Introduction to Programming and Data Structures (W1)

05 August 2023 16:57

Suppose you are playing a game in turn with the computer.
Total n number of sticks are to be picked
up in this game. Whoever picks the last one loses the game.
Neither the computer nor you can pick up
more than 3 sticks at a time. Nobody can skip a turn, i.e. at least one stick is to be picked up in a turn.
Write a program to ensure that the computer wins optimally (whenever there is a chance) irrespective of the turn.
Note that if you leave 4k + 1 sticks to the computer,the computer is guaranteed to loose the game

Solving a problem on a computer requires:
1 Understanding the logical actions (strategies) to take.
2 Choosing the best strategy.
3 Finding out the best implementation of the chosen strategy



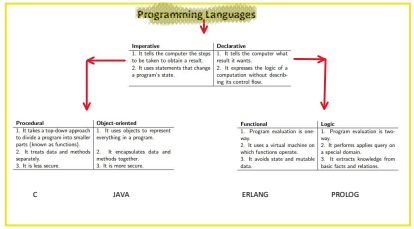
Finding the maximum – Using flowcharts



C	Python
Programming Model	
C is a **procedural programming** language	Python is an **object oriented** programming language.
Type of Language	
Middle level language as it binds the bridges between machine level and high level languages.	**high-level language** as the translation of Python code takes place into machine language, using an interpreter.
Speed	
C is a faster language compared to Python as it is compiled.	Python programs are **usually slower** than C programs as they are interpreted.
Compilation and Interpretation	
C is a **compiled programming language**. Special programs known as compilers check the C code line by line and if any error is found on any line, the program compilation stops then and there.	Python is an **interpreted programming language**. Special programs known as interpreters check the entire Python code and all the errors in the entire Python code is reported at once.
Variable Declaration	
In C, the type of the various **variables must be declared when they are created**, and only values of those particular types must be assigned to them.	In Python, **variables are untyped**, that is, there is no need to define the data type of a variable while declaring it. A given variable in Python can store values of different data types in different parts of the Python code.

Compiler	Interpreter
1. It processes the entire program at a time.	1. It processes a single instruction at a time.
2. It transforms the source code into machine readable instructions.	2. It interprets the source code into executables.
3. It generates intermediate object code, hence memory requirement is more.	3. It does not generate intermediate object code, hence memory requirement is less.
4. It is relatively faster.	4. It is relatively slower.

Programming Languages



What will be the output of the following program?

```
var1 = 123
var2 = 123
print(hex(id(var1)), hex(id(var2)))
var1 = 'Python'
var2 = 'Python'
print(hex(id(var1)), hex(id(var2)))
var1 = [1, 2, 3]
var2 = [1, 2, 3]
print(hex(id(var1)), hex(id(var2)))
```

Output:

```
0x812ee0 0x812ee0
0x9227e8 0x9227e8
0x27f9430 0x31b8740
```

Lists of sublists

What will be the output of the following program?

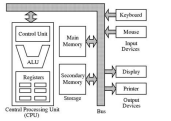
```
la1 = [[] for i in range(3)]
la1[0].append(10)
la1[1].append(20)
la1.append(30)
print(la1)
la2 = [[]]*3
la2[0].append(10)
la2[1].append(20)
la2.append(30)
print(la2)
```

```
[[10], [20], [], 30]
[[10, 20], [10, 20], [10, 20], 30]
```

Operator precedence (highest to lowest)

Associativity	Operator	Description
Left-to-right	()	Parentheses (grouping)
Left-to-right	f(args...)	Function call
Left-to-right	x[index:index]	Slicing
Left-to-right	x[index]	Array Subscription
Right-to-left	**	Exponentiation
Left-to-right	~x	Bitwise not
Left-to-right	+x -x	Positive, Negative
Left-to-right	* / %	Multiplication Division Modulo
Left-to-right	+	Addition Subtraction
Left-to-right	<< >>	Bitwise left shift Bitwise right shift
Left-to-right	&	Bitwise AND
Left-to-right	^	Bitwise XOR
Left-to-right	|	Bitwise OR
Left-to-right	in, not in, is, is not, <, <=, >, >=, <>, !=, ==	Membership Relational Equality Inequality
Left-to-right	not x	Boolean NOT
Left-to-right	and	Boolean AND
Left-to-right	or	Boolean OR
Left-to-right	lambda	Lambda expression

A digital computer – The internals



Given the number $s_k s_{k-1} \cdots s_1 s_0$ represented in base b, its decimal value is:

$$\sum_{i=0}^{k} s_i * b^i.$$

A digital computer – Binary arithmetic

Decimal 6.625 equals to 110.101 in binary.

Converting the integer part 6 to binary:

Integer	Operation	Quotient (integer)	Remainder
6	6 / 2	3	0
3	3 / 2	1	1
1	1 / 2	0 [STOP]	1

Converting the fractional part 0.625 to binary:

Fraction	Operation	Fraction	Integer
0.625	0.625 * 2	0.25	1
0.25	0.25 * 2	0.5	0
0.5	0.5 * 2	0 [STOP]	1

GIF

Introduction to Programming and Data Structures (W2)

05 August 2023 16:57

PYTHON

CHARACTERISTICS :

- Python is a free and open source language, first written in C
- Python is an interpreted language.
- Python is not a free-form language.
- Python is a strongly typed language.
- Python is an object-oriented language but it also supports procedural oriented programming.
- Python is a high level language.
- Python is a portable and cross-platform language.
- Python is an extensible language.

Data Types in Python

Type	Data Type	Description	Example
Number	Int	Integer numbers	-10, -4, 0, 120, 1034
	Float	Floating point number	-3.08, 7.0, 23.41
	Complex	Complex numbers	3 + 4j, 2 – 2i
Boolean	Bool	Boolean number	0, 1
Sequence	String	Set of characters	'hello'
	List	Set of items separated by comma	[1,2.3,'a']
	Tuple	Set of items separated by comma enclosed in (). It is not editable	(2,'b',3.4)
Missing value	None	Signifies absence of value in a situation	None
Mapping	Dictionary	Holds data items in key-value pairs	{'a':1,'b':2}

Operator	Description	a = 60 b = 13	# 60 = 0011 1100 # 13 = 0000 1101
& Binary AND	Operator copies a bit to the result if it exists in both operands	(a & b) (means 0000 1100)	
| Binary OR	It copies a bit if it exists in either operand.	(a | b) = 61 (means 0011 1101)	
^ Binary XOR	It copies the bit if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)	
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011 in 2's complement form due to a signed binary number.	
<< Binary Left Shift	The left operands value is moved left by the number of bits specified by the right operand.	a << 2 = 240 (means 1111 0000)	
>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand.	a >> 2 = 15 (means 0000 1111)	

Recursive versus iterative implementations

Let's execute the following complete program:

```
def factorial(n): # Iterative version
    prod = 1
    if n < 0:
        return -1 # Error condition
    while n:
        print('Value of n (', hex(id(n)), ') = ', n)
        prod *= n
        n = n - 1
    return prod
factorial(5)
```



Recursive versus iterative implementations

Adress keeps changing

Output:

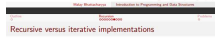
```
Value of n ( 0x7fb0a6c10170 ) =  5
Value of n ( 0x7fb0a6c10130 ) =  4
Value of n ( 0x7fb0a6c10130 ) =  3
Value of n ( 0x7fb0a6c10110 ) =  2
Value of n ( 0x7fb0a6c100f0 ) =  1
```

From mynotes + cs_screenshots

Recursive versus iterative implementations

Let's execute the following complete program:

```
def factorial(n):                   # Recursive version
    if n < 0:
        return -1                   # Error condition
    print('Value of n (', hex(id(n)), ') = ', n)
    if n == 0 or n == 1:
        return 1                    # Base case
    else:
        print ('Entering into factorial(', n-1, ')');
        return n * factorial(n-1) # Recursive call
factorial(5)
```



Recursive versus iterative implementations

Adress not changing

Output:

```
Value of n ( 0x7f0f35500170 ) =  5
Entering into factorial( 4 )
Value of n ( 0x7f0f35500150 ) =  4
Entering into factorial( 3 )
Value of n ( 0x7f0f35500130 ) =  3
Entering into factorial( 2 )
Value of n ( 0x7f0f35500110 ) =  2
Entering into factorial( 1 )
Value of n ( 0x7f0f355000f0 ) =  1
```

PYTHON MODULES

MATH MODULE FUNCTIONS:

ceil(x)	– Ceiling of x
comb(n, r)	– Number of ways to choose r from n (^nC_r)
copysign(x, y)	– Float with magnitude of x but sign of y
fabs(x)	– Absolute value of x
factorial(x)	– Factorial of x
floor(x)	– Floor of x

Table: Functions in math module

Mathematical functions for real numbers

fmod(x, y)	– x % y (preferable for integers)
frexp(x)	– Mantissa and exponent of x as a pair (m, e)
fsum(iterable)	– Accurate floating point sum of values in iterable
gcd(x, y)	– GCD of the integers x and y
isclose(x, y, *, rel_tol =1e-09, abs_tol=0.0)	– Whether x is close to y w.r.t max/min allowed tolerance
isfinite(x)	– Whether x is finite (or ∞/NaN)
isinf(x)	– Whether x is infinite
isnan(x)	– Whether x is NaN ("not a number")
isqrt(x)	– Integer square root of x
ldexp(x, i)	– $x * 2^i$

modf(x)	– Fractional and integer parts of x
perm(n, r=None)	– Number of ways to choose k from n without repetition (^nP_r)
prod(iterable, *, start=1)	– Product of values in iterable
remainder(x, y)	– Remainder of x when divided by y
trunc(x)	– Value of x truncated to an integral
exp(x)	– e^x
expm1(x)	– $e^x - 1$ (provides a better precision)
log(x[, base])	– Natural logarithm of x (base e)
log1p(x)	– Natural logarithm of 1+x (base e)
log2(x)	– Base-2 logarithm of x
log10(x)	– Base-10 logarithm of x
pow(x, y)	– x^y

sqrt(x)	– Square root of x
acos(x)	– Arc cosine of x (result in radians)
asin(x)	– Arc sine of x (result in radians)
atan(x)	– Arc tangent of x (result in radians)
atan2(x, y)	– Arc tangent of x/y (in radians)
cos(x)	– Cosine of x
dist(iterable1, iterable1)	– Euclidean distance between iterable 1 and iterable2
hypot(*coordinates)	– Euclidean norm sqrt(x*x + y*y) for the point (x, y), sqrt(sum(x**2 for x in coordinates))
sin(x)	– Sine of x
tan(x)	– Tangent of x

degrees(x)	– Convert angle x from radians to degrees
radians(x)	– Convert angle x from degrees to radians.
acosh(x)	– Inverse hyperbolic cosine of x
asinh(x)	– Inverse hyperbolic sine of x
atanh(x)	– Inverse hyperbolic tangent of x
cosh(x)	– Hyperbolic cosine of x
sinh(x)	– Hyperbolic sine of x
tanh(x)	– Hyperbolic tangent of x
erf(x)	– Error function at x
erfc(x)	– Complementary error function at x
gamma(x)	– Gamma function at x
lgamma(x)	– Natural logarithm of absolute value of Gamma function at x

pi	– π = 3.14..., to available precision
e	– e = 2.71..., to available precision
tau	– τ = 6.28..., to available precision
inf	– Floating-point positive infinity
nan	– Floating-point NaN

Mathematical functions for complex numbers

Using the cmath module:

```
import cmath
cmath.<function_name>()
```

OR

```
from cmath import <function_name>
<function_name>()
```

phase(x)	– Phase of x (in float)
polar(x)	– Representation of x in polar coordinates as (r, phi)
rect(r, phi)	– Complex number x with polar coordinates r and phi
exp(x)	– e^x
log(x[, base])	– Natural logarithm of x (base e)
log10(x)	– Base-10 logarithm of x

Mathematical functions for complex numbers

isclose(x, y, *, rel_tol =1e-09, abs_tol=0.0)	– Whether x is close to y w.r.t max/min allowed tolerance
isfinite(x)	– Whether x is finite (or ∞/NaN)
isinf(x)	– Whether x is infinite
isnan(x)	– Whether x is NaN
pi	– π = 3.14..., to available precision
e	– e = 2.71..., to available precision
tau	– τ = 6.28..., to available precision
inf	– Floating-point positive infinity
infj	– Complex number with zero real and positive infinity imaginary parts
nan	– Floating-point NaN
nanj	– Complex number with zero real part and NaN imaginary part

Mathematical functions for complex numbers

sqrt(x)	– Square root of x
acos(x)	– Arc cosine of x
asin(x)	– Arc sine of x
atan(x)	– Arc tangent of x
cos(x)	– Cosine of x
sin(x)	– Sine of x
tan(x)	– Tangent of x
acosh(x)	– Inverse hyperbolic cosine of x
asinh(x)	– Inverse hyperbolic sine of x
atanh(x)	– Inverse hyperbolic tangent of x
cosh(x)	– Hyperbolic cosine of x
sinh(x)	– Hyperbolic sine of x
tanh(x)	– Hyperbolic tangent of x

STATISTICS MODULE FUNCTIONS:

Statistical functions

Using the statistics module:

```
import statistics
statistics.<function_name>()
```

mean(X)	– Arithmetic mean of the data in X
fmean(X)	– Arithmetic mean of the data (converted to float) in X
geometric_mean(X)	– Geometric mean of the data (converted to float) in X
harmonic_mean(X)	– Harmonic mean of the data in X

Table: Functions in statistics module

median(X)	– Median of the data in X
median_low(X)	– Low median of the data in X
median_high(X)	– High median of the data in X
median_grouped(X, interval)	– Median of the grouped data in X
mode(X)	– Most frequent data item in X
multimode(X)	– Most frequent data items in the order they appear in X

pstdev(X, mu=None)	– Population standard deviation of the data in X
pvariance(X, mu=None)	– Population variance of the data in X
stdev(X, xbar=None)	– Sample standard deviation of the data in X
variance(X, xbar=None)	– Sample variance of the data in X
quantiles(X, *, n, method)	– Divide data in X into n continuous intervals with equal probability

RANDOM MODULE FUNCTIONS:

- random.shuffle(list): Shuffles the elements of a list in-place
- random.randint(a,b): Generates a random integer between a and b (inclusive)
- random.random(): Generates a random float between 0 and 1

NUMPY MODULE FUNCTIONS:

Numpy is a module of python that offers functions for fast mathematical computation on arrays. Arrays is a named group of homogeneous elements. It's very similar to lists.

- random_integers : np.random.randint(x,y,size=(a,b)) #Generates a axb array of random integers from a to b-1
- np.random.random(x): Generates array of x random floats in the half-open interval [0.0, 1.0); if (size=(a,b))# Generates a axb array of random floats between 0.0 and 1.0.
- np.random.randn(a,b) # Generates a axb array of random numbers from a standard normal distribution (mean 0, standard deviation 1)
- np.random.seed(n) # Sets the seed value to n. Seeds the random number generator for reproducibility. When using the same seed value (e.g., 42), you'll get the same random numbers every time you run the code, ensuring reproducibility.

Drawing samples from different distributions using numpy

beta(a, b[, size])	– Samples from Beta distribution
binomial(n, p[, size])	– Samples from binomial distribution
chisquare(df[, size])	– Samples from chi-square distribution
dirichlet([alpha[, size])	– Samples from Dirichlet distribution
exponential([scale, size])	– Samples from exponential distribution
f(dfnum, dfden[, size])	– Samples from F distribution
gamma(shape[, scale, size])	– Samples from Gamma distribution
geometric(p[, size])	– Samples from geometric distribution
gumbel([loc, scale, size])	– Samples from Gumbel distribution

Table: Functions in random submodule of numpy

hypergeometric(ngood, nbad, nsample[, size])	– Samples from hypergeometric distribution
laplace([loc, scale, size])	– Samples from Laplace distribution
logistic([loc, scale, size])	– Samples from logistic distribution
lognormal([mean, sigma, size])	– Samples from log-normal distribution
logseries(p[, size])	– Samples from logarithmic series distribution

multinomial(n, pvals[, size])	– Samples from multinomial distribution
multivariate_normal(mean, cov[, size, ...])	– Samples from multivariate normal distribution
negative_binomial(n, p[, size])	– Samples from negative binomial distribution
noncentral_chisquare(df, nonc[, size])	– Samples from noncentral chi-square distribution
noncentral_f(dfnum, dfden, nonc[, size])	– Samples from noncentral F distribution
normal([loc, scale, size])	– Samples from normal distribution

pareto(a[, size])	– Samples from pareto distribution
poisson([lam, size])	– Samples from Poisson distribution
power(a[, size])	– Samples from power distribution
rayleigh([scale, size])	– Samples from Rayleigh distribution
standard_cauchy([size])	– Samples from standard Cauchy distribution
standard_exponential([size])	– Samples from standard exponetial distribution
standard_gamma(shape[, size])	– Samples from standard Gamma distribution
standard_normal([size])	– Samples from standard normal distribution

standard_t(df[, size])	– Samples from standard Student's t distribution
triangular(left, mode, right[, size])	– Samples from triangular distribution
uniform([low, high, size])	– Samples from uniform distribution
vonmises(mu, kappa[, size])	– Samples from von Mises distribution
wald(mean, scale[, size])	– Samples from Wald distribution
weibull([a[, size])	– Samples from Weibull distribution
zipf([a[, size])	– Samples from Zipf distribution